```kotlin
fun main() {  new *
    for (i in 1 ≤ .. ≤ 10) {
        println("Hello, welcome to the tutorial")
    }
    TODO( reason: "Write some cool code so that they are impressed")
}
```

Programming Tutorial - 27

12. Tutorial - Tips

Kuzey Aydınhan | 13. February 2026

Contents

1. Overview

2. Tips

Starting the Exercise
Implementation
Guidelines
Turning In

Information

- First Exercise release on 16. Feb.
- First Exercise deadline: 17. March (06:00)
- Second Exercise release on 02. March
- Second Exercise deadline: 31. March (06:00)

Registration

Register for Abschlussaufgaben from 16. Feb. to 26. Feb. in Campus-System

Motivation

- Tips for Final Exercises

General Tips

- Start early
- Better to do 2-5 hours a day than 10-15 hours in short time
- Try to do both exercises

- Allowed to use exemplary solutions from exercises
- If you do, do not change `@author programmieren-team` tag
- If you add or change something → add your tag as well
  - e.g. if you add new method, in the JavaDoc of method add the tag

Starting the Exercise

- First come up with a plan
- Don't start coding directly
- Read the whole sheet and understand the core idea

- Something unclear → read once again
- Have a question → check forums
- It's okay to spend 1-2 days just understanding

Modelling

- Now go through the sheet step by step
- In each section:
  - Note down classes you might need
  - Try shaping the classes with attributes

- Make sure to create base classes if you see a common pattern

gen Erweiterung planen, die textuelle Schnittstelle der Anwendung durch eine grafische Oberfläche zu ersetzen, müssen Sie zudem auf die Trennung der Belange achten!

Task
+ status
+ subtasks

Status
open, done

A.1.1 Aufgaben

Aufgaben sind die zentralen Elemente der Anwendung. Eine Aufgabe ist zu Beginn offen und kann dann durch den Nutzer als erledigt markiert werden. Nutzer können Aufgaben anlegen, als erledigt markieren, löschen und wiederherstellen (siehe Abbildung A.1). Aufgaben können auch andere Aufgaben als Unteraufgaben hinzugefügt werden. Auch die Unteraufgaben können dementsprechend wieder Unteraufgaben beinhalten.

Implementation

Outside → In

- Start from outer parts and move into logic
- Commands
  - Add the commands
  - Come up with a way to parse them
  - Find a common pattern
  - Create the base class
- Model
  - What needs to saved
  - Classes that describe the objects in exercise
  - Check your previous notes
  - Add in attributes
  - Slowly extend to methods → don't write all the methods at once, implement when needed
- Logic (Services)
  - Implement the logic for the models first
  - Then do the user interface and parsing

Package Structure

- Don't worry about the structure in the first place
- Create all the classes
- Have something working
- Afterwards clean it up

- Try to divide it all meaningfully
- Common structure would be:
  - Commands
  - Exceptions
  - Model
  - User Interface

Problems

- Take problems into account as well
- Try testing your code after changes
- Don't make big changes at once $\rightarrow$ harder to find the problem
- If you find the time, can also write your own tests to check each component on its own

Common Way

- Go through the forum each day → check for updates
- Take 1-2 hours each day to implement another part
- Test your code

- When its working refactor it
  - Taking 20-30 min time to refactor the code you've written helps keep it clean
  - Do it either after each change or at very least once a week

- Commit and push your changes
  - This will help prevent any problems in case something happens to your pc

Guidelines

- From 20 points, around 12 is functionality (tests)
- Rest is methodology → how was it implemented

- Things you need to be careful about → Guidelines: https://sdq.kastel.kit.edu/programmieren/

Point Deductions

Common

- Unnecessary Complexity
- Visibility
- JavaDoc Trivial/Empty
- Commented Out Code
- Unused Element
- Magic Literals
- Incorrect Loop Type
- Bad Naming
- Comments
- Unreadable Code
- Enums
- IO Separation
- Big Try-Catch Blocks

Visibility

- Always check the visibilities
- Attributes should be `private`
- Use `public` only when necessary

- Additionally: If attribute can be `final`, make it `final`

Comments

- Remove TODOs and commented out codes
- Do not over-comment
- Add comments only on **complex parts** (e.g. nested loops)

- Adding too much → harder to read
- Use git commits instead of commenting-out code

Bad Code

```java
// TODO Create test cases
// TODO auto-generated method
public int multiply(int first, int second) {
    // Checkstyle doesn't let me write Multiply
    // FIXME Think of variable names
    // My tutor is stupid
    // Multiplies m1 with m2.
    return first * second;
    // return second * second;
    // return m1 * m1 * m3;
}

```

Good Code

```java
// Iterates through whole board
for (int i=0; i<board.length; i++) {
    for (int j=0; j<board[i].length; j++) { .. }
}

```

Unreadable Code

(Schwieriger Code)

- Code that is too complex, nested etc.
- Hard to understand the code
- How to avoid:
  - Put in smaller methods (preferrably)
  - Comment some of the parts
- In general: try writing 20-40 lines per method
- Avoid deep nests

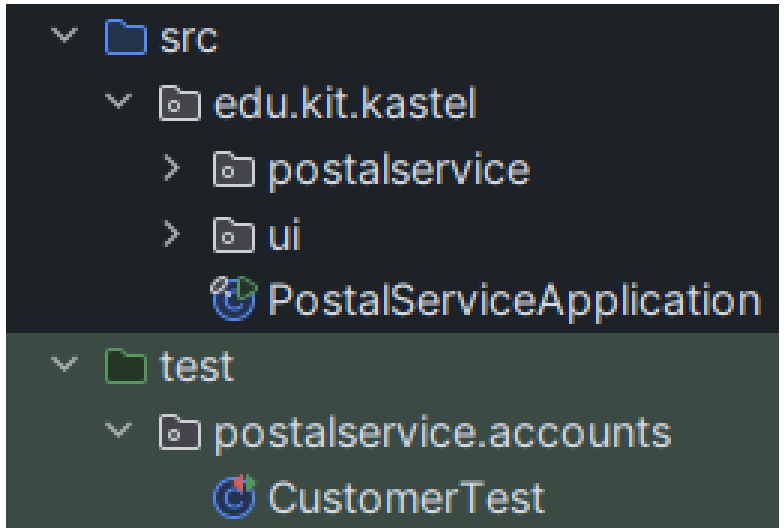
```java
public void endGame(int[] numbers, Player player) {
    int size = numbers.length;
    for (int i = 0; i < size - 1; i++) {
        int minimumIndex = i;
        for (int j = i + 1; j < size; j++) {
            if (numbers[j] < numbers[minimumIndex]) {
                minimumIndex = j;
            }
        }
        int temp = numbers[minimumIndex];
        numbers[minimumIndex] = numbers[j];
        numbers[j] = temp;
    }
    int sum = 0;
    String result = "";
    for (Integer number : numbers) {
        number += sum;
        result += number + " ";
    }
    System.out.println(result);
    System.out.println("Player " + player.getName() + " scored: " + sum);
}
```

Turning In

- You **DON'T** have to click "abgeben" to turn the exercise in
- Just like in homeworks, your latest push will be noted

- If you are writing your own tests, these must be included in `.gitignore` file

.gitignore



- Need to include the **`test/`** folder in
  » `.gitignore`

```
# ...

# Taken from https://github.com/github/gitignore

### Tests
test/

### Java
# Compiled class file
*.class
# ...

```

Conclusion

- Start early with exercises
- You are allowed to use exemplary solutions/your previous solutions
- Plan first, code afterwards
- Mind problems and bugs
- Check the guidelines
- Add tests directory in `.gitignore`

Important

Don’t forget to register for Abschlussaufgaben in Campus-System!

THANK YOU FOR YOUR ATTENTION!

Best of luck in your studies!